

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Arquitectura De Computadoras Y
Ensambladores 1
Ing. Mauricio Lopez.
Aux. José Pú.



PRACTICA 2

Javier Alejandro Mérida Gómez 202230638

Quetzaltenango, 18 de Agosto de 2026

Índice

Índice.....	2
Introducción.....	3
Análisis del problema.....	4
1. Estado Normal (Reposo).....	4
2. Prioridad Absoluta de Activación Manual.....	4
3. Evaluación Conjunta de Sensores (Lógica AND).....	4
4. Reconocimiento y Silencio de Emergencia.....	4
Implementación en Ensamblador.....	5
Identificación de entradas y salidas.....	6
Tabla de elementos.....	6
Tabla de Pruebas.....	6
Diagrama de flujo.....	7
Conclusiones.....	8
Anexos.....	9
Figura 1 - Circuito realizado en SimulideIDE.....	9
Figura 2 - Prueba 1.....	9
Figura 3 - Prueba 2.....	10
Figura 4 - Prueba 3.....	10
Figura 5 - Prueba 4.....	11
Figura 6 - Prueba 5.....	11
Figura 7 - Prueba 6.....	12
Figura 8 - Prueba 7.....	12

Introducción

El uso de microcontroladores en el desarrollo de sistemas embebidos de seguridad representa una solución eficiente y de bajo costo para el monitoreo en tiempo real de variables ambientales. El presente informe documenta el diseño, la programación en lenguaje ensamblador y la simulación de un sistema básico de alarma contra incendios implementado con el microcontrolador **PIC16F628A**.

El sistema procesa cuatro señales de entrada (activación manual, sensor de humo, sensor de temperatura y botón de silencio) para gestionar el estado de tres salidas digitales (indicadores visuales LED verde y rojo, y una alarma sonora Buzzer).

A lo largo del documento se detalla la configuración de los registros de control del microcontrolador, la estructura del código desarrollado en MPLAB X IDE v5.35 utilizando el ensamblador MPASM, y la validación funcional mediante el simulador SimulIDE.

Análisis del problema

El problema se dividió y analizó en cuatro estados lógicos principales:

1. Estado Normal (Reposo)

Cuando no se detecta ninguna amenaza, el sistema debe señalar operabilidad sin riesgo. Para ello, la lógica mantiene encendido únicamente el LED Verde, mientras que el LED Rojo y el Buzzer se mantienen desactivados.

2. Prioridad Absoluta de Activación Manual

La activación del interruptor manual representa una condición de emergencia deliberada provocada por un usuario. El análisis del problema dicta que esta entrada prevalece sobre cualquier otra condición o estado previo.

- Si manual es activo, de inmediato se apaga el LED Verde, se enciende el LED Rojo y se activa el Buzzer.
- Esta condición desactiva la función de silencio, garantizando que la alarma sonora permanezca encendida ininterrumpidamente mientras la palanca o interruptor manual continúe activo.

3. Evaluación Conjunta de Sensores (Lógica AND)

Para evitar falsas alarmas provocadas por fallas en un solo sensor o fluctuaciones ambientales aisladas, el enunciado establece una condición de coincidencia entre el sensor de humo y el de temperatura.

- Si solo uno de los sensores se activa, el sistema descarta la alerta y permanece en Estado Normal.
- Si ambos sensores se activan simultáneamente, se confirma la condición de incendio, activando el LED Rojo y el Buzzer.

4. Reconocimiento y Silencio de Emergencia

Cuando la alarma es desencadenada por los sensores, se habilita el uso del botón de silencio.

- Esta función permite apagar temporalmente la señal audible (Buzzer = OFF) para reducir la contaminación acústica durante la gestión de la emergencia.
- No obstante, la advertencia visual permanece activa (LED Rojo = ON) como indicador de que el riesgo latente aún no se ha despejado.

Implementación en Ensamblador

Para ejecutar esta lógica, el código hace uso del método de sondeo continuo (Polling) dentro de un ciclo principal (MAIN_LOOP). Mediante las instrucciones de salto condicional BTFSC (*Bit Test File, Skip if Clear*) y BTFSS (*Bit Test File, Skip if Set*), la CPU evalúa bit a bit el estado del puerto de entrada (PORTB) y desvía el flujo del programa hacia las subrutinas de respuesta correspondiente (NORMAL, ALARMA_MANUAL, ALARMA_SENSOR y SIL_ON).

Identificación de entradas y salidas

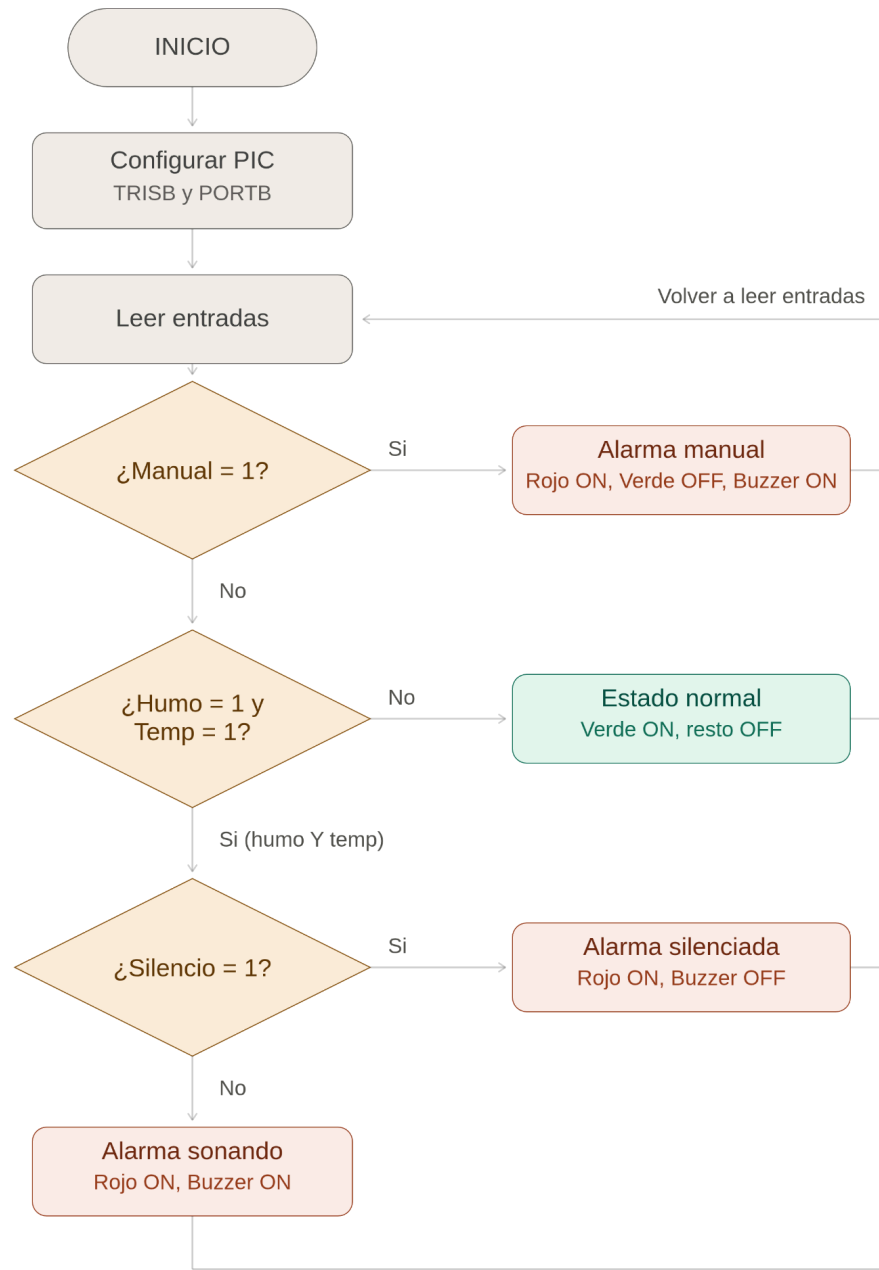
Tabla de elementos

Elemento	Tipo	Puerto en Código	Pin Físico del PIC16F628A
Activación manual	Entrada	PORTB,0 (RB0)	Pin 6
Sensor de humo	Entrada	PORTB,1 (RB1)	Pin 7
Sensor de temperatura	Entrada	PORTB,2 (RB2)	Pin 8
Silencio (Reconocimiento)	Entrada	PORTB,3 (RB3)	Pin 9
LED verde	Salida	PORTB,4 (RB4)	Pin 10
LED rojo	Salida	PORTB,5 (RB5)	Pin 11
Buzzer	Salida	PORTB,6 (RB6)	Pin 12

Tabla de Pruebas

No.	M	H	T	S	LED VERDE	LED ROJO	BUZZZER
1	0	0	0	0	ON	OFF	OFF
2	1	0	0	0	OFF	ON	ON
3	0	1	0	0	ON	OFF	OFF
4	0	0	1	0	ON	OFF	OFF
5	0	1	1	0	OFF	ON	ON
6	0	1	1	1	OFF	ON	OFF
7	1	1	1	1	OFF	ON	ON

Diagrama de flujo



Conclusiones

- Sincronización Hardware-Software: La correcta operación de un microcontrolador depende en igual medida del código fuente y de la arquitectura física en la simulación. La omisión de señales clave como la línea de Reset (MCLR a VDD) o un desajuste entre los pines configurados en el registro TRISB y los cables del circuito impiden el funcionamiento del sistema, demostrando que el diseño embebido exige una validación integral.
- Efectividad del Lenguaje Ensamblador: La programación en ensamblador permite un control determinista de la CPU del PIC16F628A. El uso de instrucciones como BTFSC y BTFSS ofrece una forma altamente eficiente de evaluar estructuras de decisión en tiempo real con una latencia mínima, ideal para sistemas críticos como alarmas de incendio.
- Validación por Simulación: El uso de herramientas como SimulIDE facilita la detección oportuna de inconsistencias en la lógica de control y fallas en las líneas de nivel lógico (como entradas flotantes), permitiendo corroborar el comportamiento del sistema ante todas las combinaciones posibles de la tabla de verdad antes de su implementación en un protoboard físico.

Anexos

Figura 1 - Circuito realizado en SimulideIDE

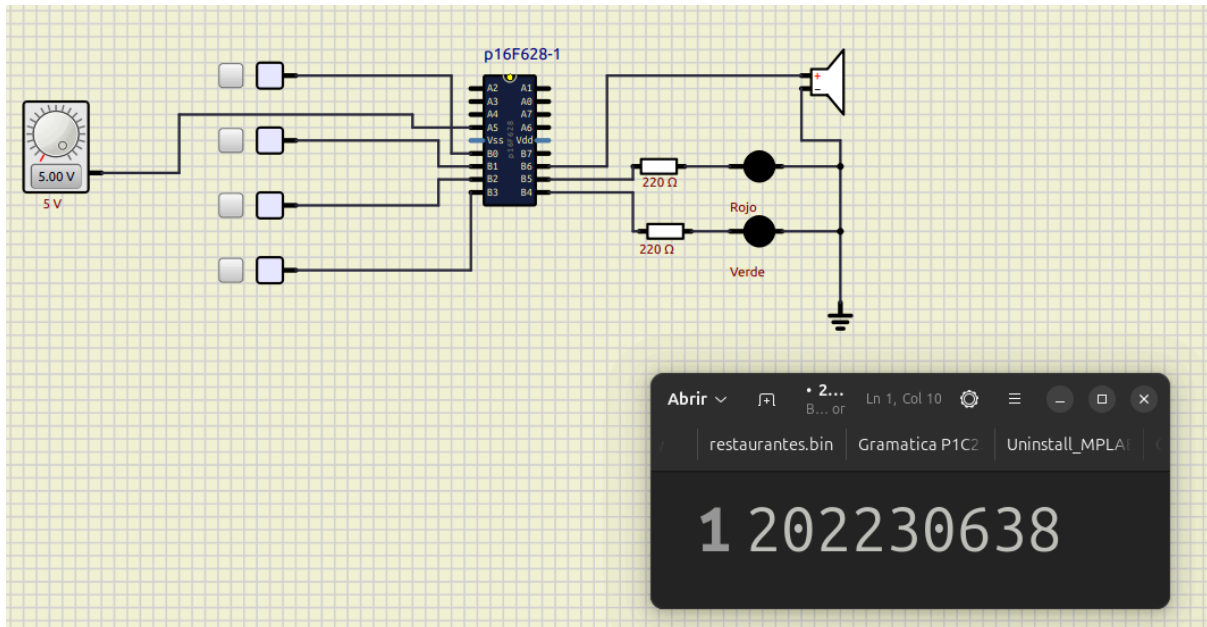


Figura 2 - Prueba 1

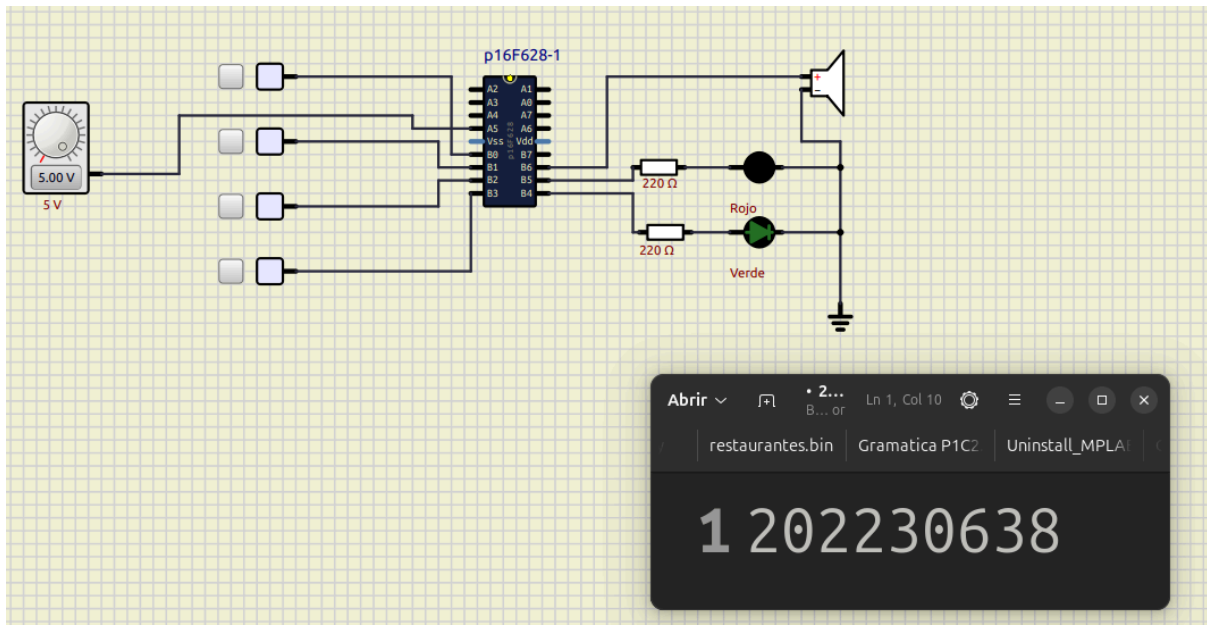


Figura 3 - Prueba 2

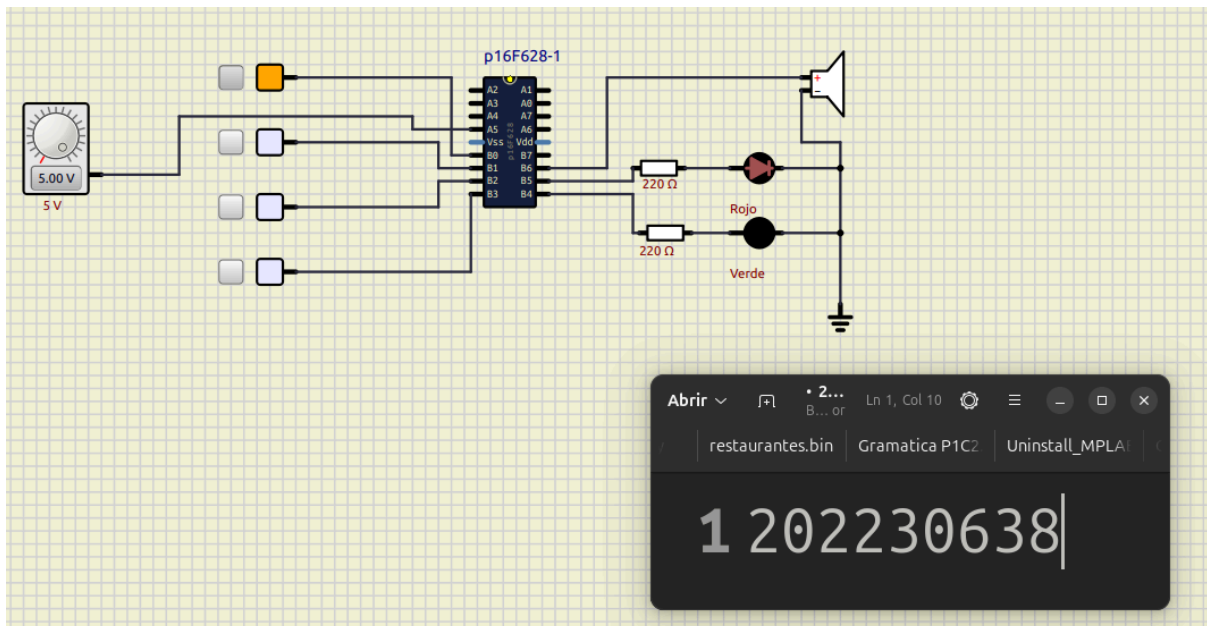


Figura 4 - Prueba 3

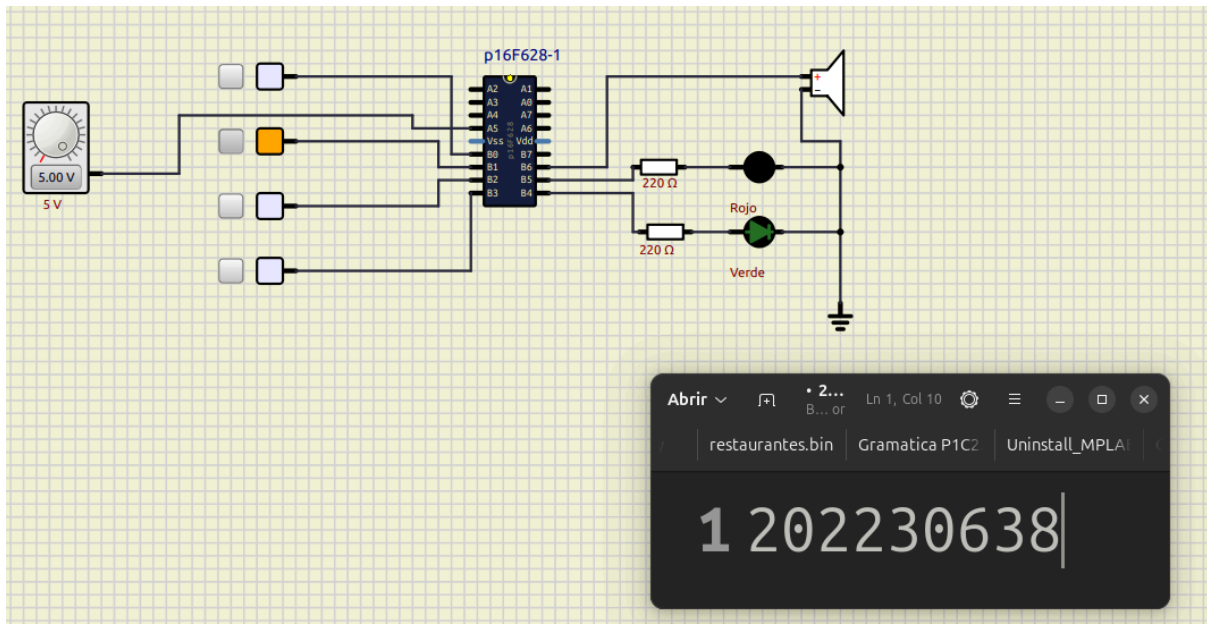


Figura 5 - Prueba 4

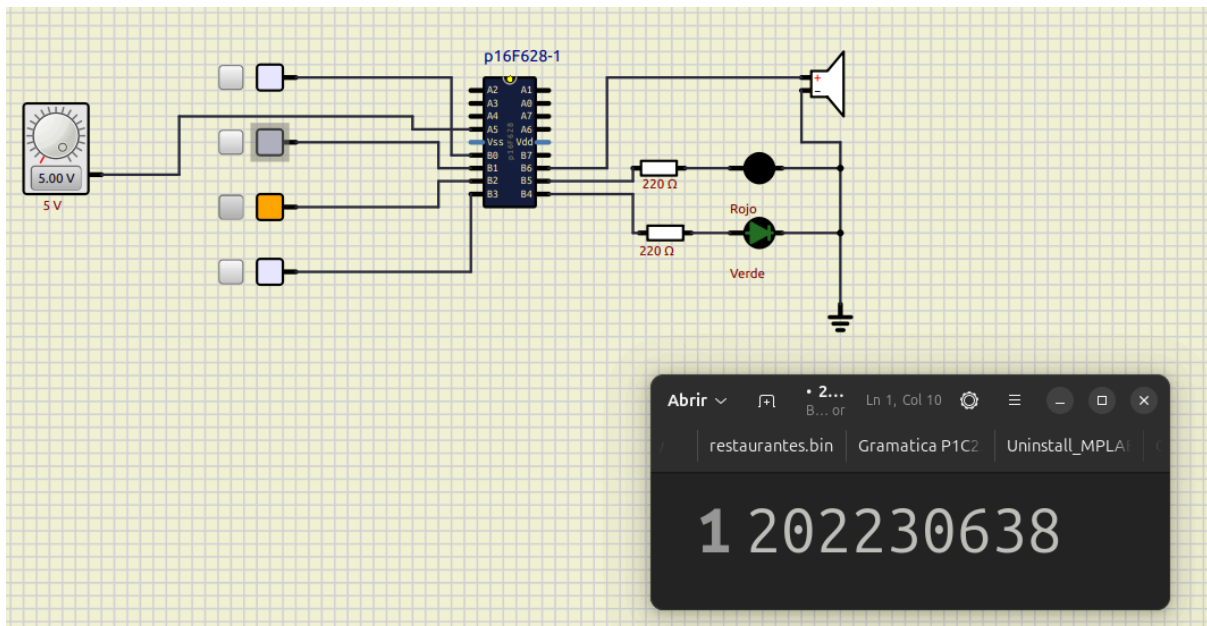


Figura 6 - Prueba 5

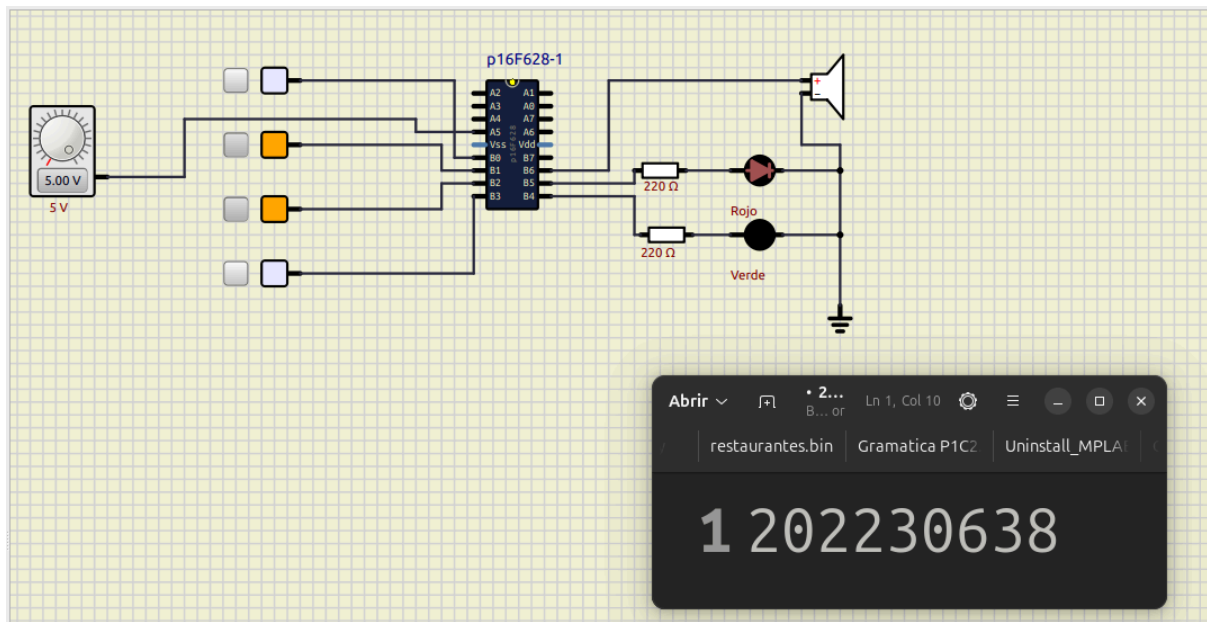


Figura 7 - Prueba 6

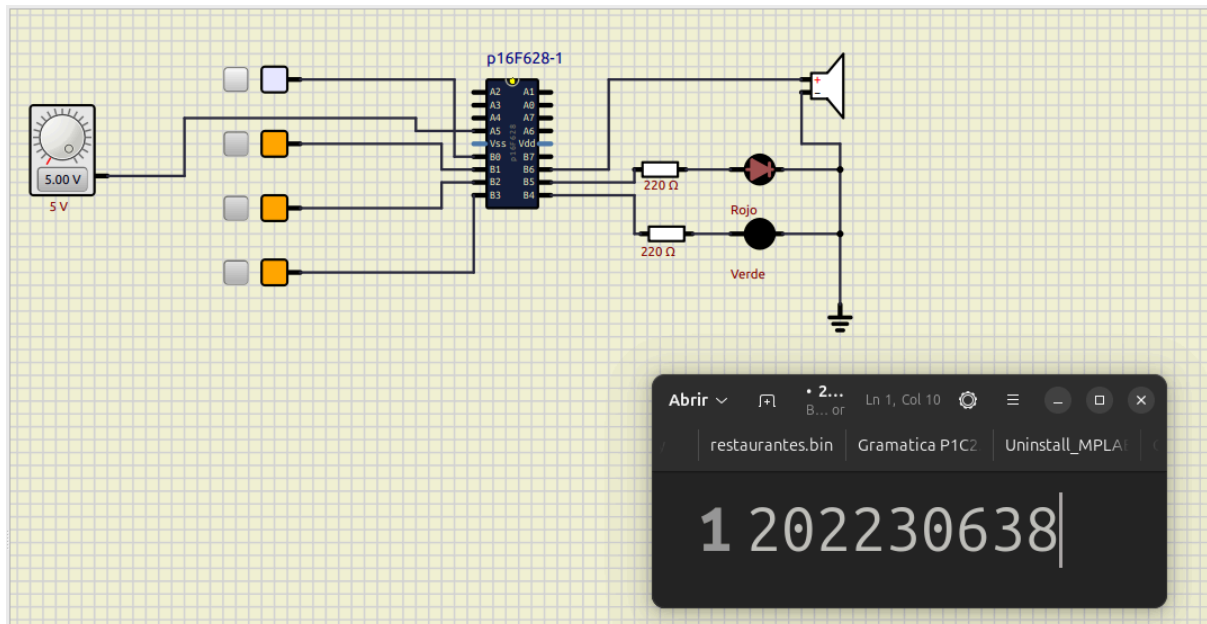


Figura 8 - Prueba 7

